Our Prototype, Explained Line by Line

eBPF-Based Bufferbloat Detection + Adaptive AQM

What the project does, and what every line of code does, in plain language

CSC 303 Computer Networks | Prototype stage

1. The project in two minutes

The problem

You are playing an online game. Your ping is 20 ms, so the game feels instant. Then someone on the same Wi-Fi starts a big download. Your ping jumps to 1000 ms and the game becomes unplayable. The strange part: the internet is not broken and the download is running at full speed.

Here is why. Data travels in small pieces called **packets**. The router can only send one packet at a time, so packets that have to wait stand in a line called a **queue**. A download keeps sending faster and faster until the queue is completely full. Your tiny game packet then has to stand at the back of a very long line. This problem is called **bufferbloat**.

Picture it like this. A highway narrows into one toll booth. Cars wait in a parking area before the booth. Making the parking area huge does not let more cars through per minute. It only means each car waits longer. A big download fills the whole parking area with trucks, and your motorbike (the game packet) is stuck behind all of them.

The fix that already exists

Linux lets you choose the **rule** the queue follows. The simple rule is called `pfifo`: first come, first served, and only throw packets away when the line is totally full. Smarter rules exist, such as `fq_codel` and `cake`. They give each conversation its own line, and they politely tell the download to slow down before the line gets long. We did **not** invent these. They are already inside Linux.

What we built

A program that watches the queue, notices by itself when bufferbloat starts, switches the queue to a smarter rule, and then checks that the switch really helped. Five steps, repeated forever:

```
   MEASURE  ->  DETECT  ->  DECIDE  ->  ADAPT  ->  VERIFY
      ^                                             |
      +---------------------------------------------+

 MEASURE  how long are packets waiting?         (eBPF program inside Linux)
 DETECT   is the line long, and staying long?   (controller.py)
 DECIDE   should we change the queue rule?      (controller.py)
 ADAPT    change it                             (the Linux "tc" command)
 VERIFY   measure again: did it help?           (back to MEASURE)
```

Like a thermostat. A thermostat reads the temperature, compares it with the target, switches the heater on or off, and reads the temperature again. Our program does the same thing, except the "temperature" is how long packets wait, and the "heater switch" is the queue rule.

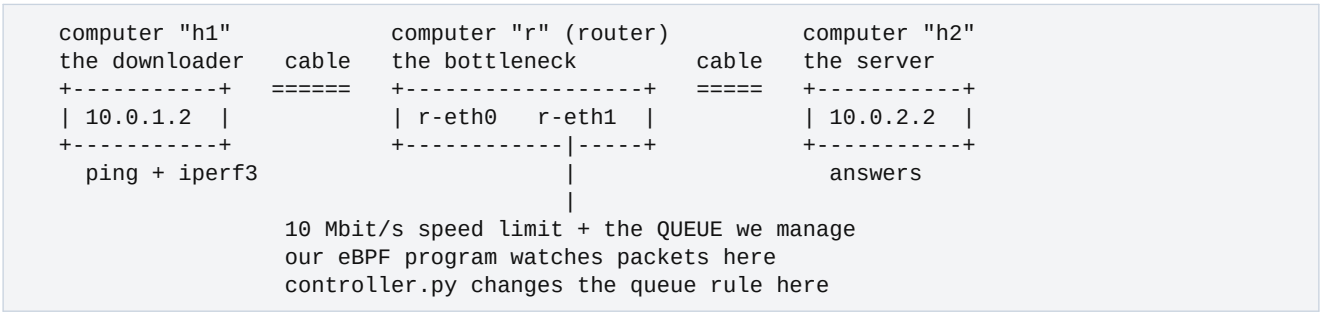
2. Words you need to know

The code uses these words all the time. Read this table once, then come back to it whenever a word in the walkthrough is unclear.

Word	What it means in simple words
packet	A small piece of data, like one envelope. A file is chopped into thousands of packets.
queue / buffer	The waiting line of packets inside a router or computer.
bufferbloat	The waiting line is so long that every packet is delayed a lot, even though nothing is broken.
latency, RTT, ping	How long a message takes to go there and come back. Measured in milliseconds (ms). 1000 ms = 1 second. Low is good.
bandwidth, Mbit/s	How much data the link can carry each second. Our test link carries 10 Mbit/s.
bottleneck	The slowest link on the path. The queue builds up just before it, like cars before a toll booth.
TCP	The system most downloads use. It keeps speeding up until packets get lost, then slows down.
flow	One conversation between two programs, for example one download or one video call.
kernel	The core of Linux. It handles all packets. Normal programs live outside it and must ask it for things.
qdisc	Short for queueing discipline: the rule the kernel uses for one waiting line. This is what we switch.
pfifo	The simplest qdisc. One line, first come first served. Causes bufferbloat when the line is long.
fq_codel	A smart qdisc. Separate line per flow, and it drops a packet early to tell a download to slow down.
cake	An even more complete smart qdisc. The strongest rule on our list.
AQM	Active Queue Management: the family name for smart qdiscs like fq_codel and cake.
tc	The Linux command (traffic control) used to look at qdiscs and change them.
eBPF	A way to put a tiny, safety-checked program inside the kernel so it can watch packets very cheaply.
BCC	A Python library that loads eBPF programs for us.
tracepoint	A marked spot inside the kernel where an eBPF program is allowed to listen.
map	A table the eBPF program writes into and our Python program reads from.
sojourn time	How long one packet actually sat in the queue: time it left minus time it joined.
namespace	A pretend separate computer inside one Linux machine. We use three of them.
veth	A pretend network cable joining two namespaces.
htb	A qdisc that limits speed. We use it to make our pretend link exactly 10 Mbit/s.
netem	A qdisc that adds fake delay, so our pretend link feels like a real 20 ms internet path.
iperf3	A tool that sends data as fast as it can. It plays the part of the big download.

3. How the files fit together

The whole prototype is six small files. Everything runs on one Linux machine, which pretends to be three computers joined by cables.



File	Lines	Job, in one sentence
`setup.sh`	12	Installs the tools we need. You run it once.
`testbed.sh`	71	Builds the three pretend computers, the cables and the slow link.
`observer.bpf.c`	138	The eBPF program. Lives inside the kernel and times every packet in the queue.
`controller.py`	372	The brain. Reads the measurements, decides, and switches the queue rule.
`run_demo.sh`	41	Runs the whole experiment twice: brain off, then brain on.
`plot.py`	102	Turns the recorded numbers into a table and a picture.

In the next sections each file is shown in small pieces. The grey box is the real code with its real line numbers. The bullet points under it say what those lines do. Every line of every file is covered.

Two languages to expect. Files ending in `.sh` are shell scripts: lists of commands you could also type by hand in a terminal. Files ending in `.py` are Python. The file ending in `.c` is C, the language the Linux kernel itself is written in. A line starting with `#` (or `//` in C) is a **comment**: a note for humans that the computer ignores.

4. setup.sh - install the tools

Run once on a new Linux machine. It downloads the programs the other files depend on.

```
#!/usr/bin/env bash
# One-time dependency install on Debian 12 / Ubuntu 22.04+ (run inside the Linux VM).
# plot.py additionally needs matplotlib, on whichever machine you plot on.
```

- `#!/usr/bin/env bash` tells Linux "run this file with the program called bash". Every shell script starts this way.
- The next two lines are comments. They say the script is for Debian or Ubuntu Linux, and that drawing the graph needs one more tool (matplotlib).

```
set -euo pipefail
export DEBIAN_FRONTEND=noninteractive
sudo apt-get update -qq
```

- `set -euo pipefail` is a safety switch: if any command fails, stop right away instead of carrying on with a half-broken setup.
- `export DEBIAN_FRONTEND=noninteractive` tells the installer not to stop and ask questions.
- `sudo apt-get update -qq` refreshes the list of software available to install. `sudo` means "do this as the administrator". `-qq` means "be quiet".

```
sudo apt-get install -y -qq --no-install-recommends iproute2 ethtool iperf3 iputils-ping \
    python3-bpfcc "linux-headers-$(uname -r)"
```

- Installs the tools. `-y` answers "yes" automatically. The backslash at the end of the line means the command continues on the next line.
- `iproute2` gives us the `ip` and `tc` commands. `ethtool` changes network card settings. `iperf3` is our fake download. `iputils-ping` is `ping`.
- `python3-bpfcc` is BCC, the library that loads eBPF programs. `linux-headers-$(uname -r)` is a description of this exact kernel, which BCC needs. `$(uname -r)` is replaced by the kernel version.

```
# sch_cake / sch_netem live in modules-extra on some Ubuntu kernels.
sudo apt-get install -y -qq "linux-modules-extra-$(uname -r)" 2>/dev/null || true
sudo modprobe -a sch_htb sch_netem sch_fq_codel sch_cake
```

- On some Ubuntu versions the `cake` and `netem` queue rules come in a separate package. This line tries to install it. `2>/dev/null || true` means: hide error messages, and do not stop if it is not there.
- `sudo modprobe -a ...` loads the queue rules into the running kernel so they are ready to use. `sch_` is short for scheduler.

```
echo "ok: kernel $(uname -r), $(python3 -c 'import bcc; print("bcc", bcc.__version__)')"
```

- Prints one line saying everything worked, with the kernel version and the BCC version. `echo` means "print this".

5. testbed.sh - build the pretend network

We cannot test on the real internet, because we would not control it. So this file builds a tiny network inside one machine where **we** create the traffic jam on purpose. That way we know exactly where the queue is.

```
 1  #!/usr/bin/env bash
 2  # Three-namespace testbed with one deliberate bottleneck.
 3  #
 4  #   bb-h1 (10.0.1.2) ---- bb-r ---- bb-h2 (10.0.2.2)
 5  #          h1-eth0   r-eth0  r-eth1   h2-eth0
 6  #
 7  #   r-eth1 : htb 10 Mbit + deep pfifo leaf   <- the queue we observe and manage
 8  #   h2-eth0: netem 20 ms on the return path  <- baseline RTT (never on the bottleneck)
 9  #
10  # Usage: sudo ./testbed.sh up|down|status
```

- Line 1 says "run with bash". Lines 2 to 11 are comments with a small drawing of the network: computer h1, then the router r, then computer h2.
- The comments also say where the two special things sit: the speed limit and queue on `r-eth1`, and the fake 20 ms delay on `h2-eth0`.
- The last comment shows how to use the file: `up` builds the network, `down` removes it, `status` shows the queue.

```
11  set -euo pipefail
```

- The same safety switch as before: stop at the first error.

```
13  RATE="${RATE:-10mbit}"      # bottleneck capacity
14  DELAY="${DELAY:-20ms}"      # emulated path delay (baseline RTT)
15  LIMIT="${LIMIT:-1000}"      # pfifo depth in packets (~1.2 s at 10 Mbit)
```

- Three settings. `RATE` is the speed of the slow link (10 Mbit/s). `DELAY` is the fake travel time (20 ms). `LIMIT` is how many packets the simple queue may hold (1000).
- `${RATE:-10mbit}` means "use the value of RATE if someone gave one, otherwise use 10mbit".
- 1000 packets at 10 Mbit/s is about 1.2 seconds of waiting. That is a badly oversized queue, on purpose.

```
17  H1=bb-h1; R=bb-r; H2=bb-h2
```

- Names for the three pretend computers, so we do not have to type them again and again. `bb` is short for bufferbloat.

```
19  down() {
20      for ns in $H1 $R $H2; do ip netns del "$ns" 2>/dev/null || true; done
21  }
```

- Defines a helper called `down`. It deletes the three pretend computers.
- `for ns in ...; do ...; done` repeats a command once for each name. `ip netns del` deletes a namespace. `2>/dev/null || true` means "if it did not exist, that is fine".

```
23  up() {
24      down
25      modprobe -a sch_htb sch_netem sch_fq_codel sch_cake 2>/dev/null || true
```

- Defines a helper called `up` that builds everything. It runs until the closing `}` further down.
- First it calls `down`, so we always start from a clean state.
- `modprobe -a ...` loads the four queue rules we need into the kernel: `htb` (speed limit), `netem` (fake delay), `fq_codel` and `cake` (the smart rules).

```
27      for ns in $H1 $R $H2; do ip netns add "$ns"; ip -n "$ns" link set lo up; done
```

- Creates the three pretend computers. `ip netns add` makes a new namespace.
- `ip -n "$ns" link set lo up` switches on each one's internal loopback connection. `-n` means "do this inside that namespace".

```
29      ip link add h1-eth0 netns $H1 type veth peer name r-eth0 netns $R
30      ip link add h2-eth0 netns $H2 type veth peer name r-eth1 netns $R
```

- Creates the two pretend cables. A `veth` cable always has two ends.
- First cable: one end called `h1-eth0` goes into h1, the other end `r-eth0` goes into the router.
- Second cable: `h2-eth0` goes into h2, and `r-eth1` goes into the router. `r-eth1` **is the important one.** It is the router's exit toward h2, and it is where the queue will be.

```
32      ip -n $H1 addr add 10.0.1.2/24 dev h1-eth0
33      ip -n $R  addr add 10.0.1.1/24 dev r-eth0
34      ip -n $R  addr add 10.0.2.1/24 dev r-eth1
35      ip -n $H2 addr add 10.0.2.2/24 dev h2-eth0
```

- Gives each cable end an address, like a house number. h1 is `10.0.1.2`, h2 is `10.0.2.2`, and the router has one address on each side.
- `/24` means "the first three numbers name the street". So h1 and h2 are on different streets, and the router joins them.

```
37      ip -n $H1 link set h1-eth0 up; ip -n $R link set r-eth0 up
38      ip -n $R  link set r-eth1 up;  ip -n $H2 link set h2-eth0 up
```

- Switches all four cable ends on. A new network interface starts switched off.

```
40      ip -n $H1 route add default via 10.0.1.1
41      ip -n $H2 route add default via 10.0.2.1
42      ip netns exec $R sysctl -qw net.ipv4.ip_forward=1
```

- Tells h1 and h2: "to reach anything not on your own street, hand it to the router".
- `sysctl -qw net.ipv4.ip_forward=1` tells the router it is allowed to pass packets from one side to the other. Linux refuses to do that unless you switch it on.

```
44      # Without this, 64 KB TSO super-packets cross the "link" and a 1000-packet
45      # queue stops meaning 1000 x 1500 bytes.
46      ip netns exec $H1 ethtool -K h1-eth0 tso off gso off gro off >/dev/null
47      ip netns exec $H2 ethtool -K h2-eth0 tso off gso off gro off >/dev/null
48      ip netns exec $R  ethtool -K r-eth0  tso off gso off gro off >/dev/null
49      ip netns exec $R  ethtool -K r-eth1  tso off gso off gro off >/dev/null
```

- A fix for a trick modern Linux plays. To save effort it glues many packets into one giant 64 KB packet (called TSO, GSO and GRO).
- That would ruin our experiment: "1000 packets" would no longer mean 1000 normal-sized packets. So `ethtool -K ... off` switches the gluing off on all four cable ends.
- `>/dev/null` hides the output of each command.

```
51      # Bottleneck: shaper on top, the managed queue as its leaf.
52      ip netns exec $R tc qdisc add dev r-eth1 root handle 1: htb default 1
53      ip netns exec $R tc class add dev r-eth1 parent 1: classid 1:1 htb rate "$RATE" ceil "$RATE"
54      ip netns exec $R tc qdisc add dev r-eth1 parent 1:1 handle 10: pfifo limit "$LIMIT"
```

- **The heart of the testbed.** Three `tc` commands on the router's exit, `r-eth1`.
- First: attach `htb`, the speed limiter, at the very top (`root`) and call it `1:`.
- Second: create a lane inside it, `1:1`, limited to 10 Mbit/s. `rate` and `ceil` being equal means "never faster than this".
- Third: hang a simple `pfifo` queue of 1000 packets under that lane. **This is the queue our controller will watch and replace.** The speed limiter stays; only this part gets swapped.

```
56      # Path delay, on the reverse direction only (ACKs and echo replies).
57      ip netns exec $H2 tc qdisc add dev h2-eth0 root netem delay "$DELAY" limit 10000
```

- Adds 20 ms of fake travel time using `netem`, so the link feels like a real internet path. Without it, everything on one machine would take almost 0 ms.
- It is placed on h2's side, the return direction, on purpose. The slow link's queue stays clean, so any extra waiting we measure there is real queueing.
- `limit 10000` gives the delay box plenty of room so that it never drops packets itself.

```
59      ip netns exec $H1 ping -c 2 -W 2 -q 10.0.2.2 >/dev/null && echo "testbed up: $RATE bottleneck on r-eth1, $DE
      > LAY baseline RTT"
60  }
```

- A quick test: h1 sends 2 pings to h2. `ip netns exec` means "run this command inside that pretend computer".
- `&&` means "only if that worked": print a message saying the testbed is up. The `}` ends the `up` helper.

```
62  status() {
63      ip netns exec $R tc -s qdisc show dev r-eth1
64  }
```

- A helper called `status` that prints the queue on `r-eth1` with its counters (`-s` means statistics). Handy to see which queue rule is installed right now.

```
66  case "${1:-}" in
67      up) up ;;
68      down) down; echo "testbed down" ;;
69      status) status ;;
70      *) echo "usage: $0 up|down|status" >&2; exit 1 ;;
71  esac
```

- The part that actually runs. `$1` is the first word you typed after the file name.
- `up` runs the `up` helper. `down` runs `down` and prints a message. `status` runs `status`.
- `*)` means "anything else": print how to use the file and stop with an error (`exit 1`).

6. observer.bpf.c - the stopwatch inside the kernel

This is the eBPF program. It is the only code of ours that runs **inside** the Linux kernel. Its job is to measure how long each packet waits in the queue.

> **Picture it like this.** Stand at the entrance of a theme-park ride and write the time on each person's hand. Stand at the exit, read the hand, and subtract. Now you know exactly how long that person queued. This program does that for every packet, about a thousand times per second, and only writes down a small summary every tenth of a second.

It listens at two spots in the kernel. `net_dev_queue` fires when a packet is handed to the queue (the entrance). `net_dev_start_xmit` fires when the packet comes out and is about to be sent (the exit).

```
 1  // eBPF observer for one egress interface (loaded by controller.py through BCC).
 2  //
 3  //   net:net_dev_queue      packet handed to the qdisc   -> stamp it, count its flow
 4  //   net:net_dev_start_xmit packet leaves the qdisc      -> sojourn = now - stamp
 5  //
 6  // Sojourn time is therefore measured per packet, for whichever qdisc is
 7  // installed (pfifo included), and aggregated in the kernel into 100 ms slots.
 8  // User space only ever reads the slots and the per-flow table.
 9  //
10  // NAME_LO/HI and MASK_LO/HI (the interface name as two u64s), WINDOW_NS and
11  // NSLOTS are passed in as -D flags.
```

- All comments. They explain the two listening spots and the idea: stamp at the entrance, subtract at the exit.
- They also say this works for **any** queue rule, even plain `pfifo`, and that our Python program only ever reads summaries, never single packets.
- The last comment lists values Python fills in before loading: the name of the interface to watch, the window length, and the number of slots.

```
13  #include <linux/skbuff.h>
14  #include <linux/netdevice.h>
15  #include <linux/ip.h>
16  #include <linux/in.h>
17  #include <linux/if_ether.h>
```

- `#include` brings in descriptions written by the Linux developers, so our code knows what things look like.
- `skbuff.h` describes how the kernel stores a packet. `netdevice.h` describes a network interface. `ip.h` describes the address label on a packet. `in.h` and `if_ether.h` give names for packet types such as TCP and UDP.

```
19  struct flow_key {
20      u32 saddr;
21      u32 daddr;
22      u16 sport;
23      u16 dport;
24      u32 proto;
25  };
```

- A `struct` is a form with named boxes. This one identifies a flow (one conversation).
- `saddr` and `daddr` are the sender and receiver addresses. `sport` and `dport` are the port numbers, which say which program on each computer. `proto` says the type (TCP, UDP or ping).
- `u32` means a whole number stored in 32 bits. `u16` is 16 bits. `u64` is 64 bits (a very big number).

```
27  struct flow_val {
28      u64 packets;
29      u64 bytes;
30      u64 last_ns;
31  };
```

- What we remember about each flow: how many packets, how many bytes, and when we last saw it (`_ns` means in nanoseconds, billionths of a second).

```
33  struct slot {
34      u64 epoch;      // which WINDOW_NS interval this slot currently holds
35      u64 count;
36      u64 sum_ns;
37      u64 min_ns;
38      u64 max_ns;
39  };
```

- The summary for one tenth of a second. `epoch` says which tenth-of-a-second this is. `count` is how many packets left. `sum_ns` is all their waiting times added up. `min_ns` is the shortest wait and `max_ns` the longest.
- With count and sum we can work out the average later. The **minimum** is the most important number, as the controller section explains.

```
41  BPF_TABLE("lru_hash", u64, u64, enq_ts, 16384);                      // skb -> enqueue time
42  BPF_TABLE("lru_hash", struct flow_key, struct flow_val, flows, 4096); // five-tuple -> counters
43  BPF_ARRAY(slots, struct slot, NSLOTS);
```

- Creates the three **maps**: tables that live in the kernel and that Python can read.
- `enq_ts` is the "time written on the hand": for each packet, the time it entered the queue. Room for 16384 packets.
- `flows` holds the per-flow counters, up to 4096 flows.
- `lru_hash` means: when the table is full, throw out the entry that has not been used for the longest time. So the table can never overflow.
- `slots` is a simple numbered list of the tenth-of-a-second summaries.

```
45  static __always_inline int on_target(struct sk_buff *skb)
46  {
47      struct net_device *dev = NULL;
48      u64 name[2] = {};
49
50      bpf_probe_read_kernel(&dev, sizeof(dev), &skb->dev);
51      if (!dev)
52          return 0;
53      bpf_probe_read_kernel(&name, sizeof(name), &dev->name);
54      return (name[0] & MASK_LO) == NAME_LO && (name[1] & MASK_HI) == NAME_HI;
55  }
```

- A helper that answers one question: "is this packet leaving through the interface we care about?" The kernel handles packets for every interface, and we only want `r-eth1`.
- `bpf_probe_read_kernel` is the only safe way an eBPF program may read kernel memory. First we read which interface the packet belongs to (`skb->dev`). `skb` is the kernel's word for a packet.
- `if (!dev) return 0;` means: if there is no interface, answer "no" and stop.
- Then we read the interface's name as two big numbers and compare them with the name Python gave us. Comparing two numbers is much faster than comparing text letter by letter.
- `&&` means "and". The answer is yes only if both halves of the name match.

```
57  static __always_inline void count_flow(struct sk_buff *skb, u32 len, u64 now)
58  {
59      unsigned char *head = NULL;
60      u16 nh_off = 0, proto = 0;
61      struct iphdr ip = {};
62      struct flow_key key = {};
63      struct flow_val zero = {}, *val;
```

- A helper that works out which flow a packet belongs to and adds it to that flow's counters.
- The first lines create empty variables to fill in. `= {}` and `= 0` mean "start empty".

```
65      bpf_probe_read_kernel(&proto, sizeof(proto), &skb->protocol);
66      if (proto != htons(ETH_P_IP))
67          return;
68      bpf_probe_read_kernel(&head, sizeof(head), &skb->head);
69      bpf_probe_read_kernel(&nh_off, sizeof(nh_off), &skb->network_header);
70      if (bpf_probe_read_kernel(&ip, sizeof(ip), head + nh_off) < 0)
71          return;
```

- Reads the packet's type. If it is not a normal internet (IPv4) packet, `return` leaves the helper without counting it.
- Then it finds where the address label starts inside the packet (`head` plus `nh_off`) and copies that label into `ip`. If the copy fails (`< 0`), leave.

```
73      key.saddr = ip.saddr;
74      key.daddr = ip.daddr;
75      key.proto = ip.protocol;
76      if (ip.protocol == IPPROTO_TCP || ip.protocol == IPPROTO_UDP) {
77          u16 ports[2] = {};
78          bpf_probe_read_kernel(&ports, sizeof(ports), head + nh_off + ip.ihl * 4);
79          key.sport = ntohs(ports[0]);
80          key.dport = ntohs(ports[1]);
81      }
```

- Fills in the form: sender address, receiver address and type.
- If the packet is TCP or UDP, also read the two port numbers. They sit right after the address label, and `ip.ihl * 4` is the length of that label.
- `ntohs` fixes the byte order of the port numbers so they read as normal numbers.

```
83      val = flows.lookup_or_try_init(&key, &zero);
84      if (!val)
85          return;
86      __sync_fetch_and_add(&val->packets, 1);
87      __sync_fetch_and_add(&val->bytes, len);
88      val->last_ns = now;
89  }
```

- Looks this flow up in the `flows` table. If it is new, create it with zeros.
- `if (!val) return;` means: if that failed, give up quietly.
- `__sync_fetch_and_add` adds safely even if two processor cores do it at the same moment. We add 1 packet and `len` bytes, then note the time.

```
 91  TRACEPOINT_PROBE(net, net_dev_queue)
 92  {
 93      struct sk_buff *skb = (struct sk_buff *)args->skbaddr;
 94      u64 key = (u64)skb, now;
 95
 96      if (!on_target(skb))
 97          return 0;
 98      now = bpf_ktime_get_ns();
 99      enq_ts.update(&key, &now);
100      count_flow(skb, args->len, now);
101      return 0;
102  }
```

- **The entrance.** This block runs every time any packet is handed to a queue.
- `args->skbaddr` is where the packet sits in memory. We use that location as the packet's name tag (`key`).
- If the packet is not for our interface, stop (`return 0`).
- `bpf_ktime_get_ns()` reads the kernel's clock. `enq_ts.update` writes that time next to the packet's name tag. This is "writing the time on the hand".
- Finally `count_flow` adds the packet to its flow's counters.

```
104  TRACEPOINT_PROBE(net, net_dev_start_xmit)
105  {
106      u64 key = (u64)args->skbaddr, now, sojourn, epoch;
107      u64 *tsp = enq_ts.lookup(&key);
108      struct slot *s;
109      int idx;
110
111      if (!tsp)
112          return 0;
113      now = bpf_ktime_get_ns();
114      sojourn = now - *tsp;
115      enq_ts.delete(&key);
```

- **The exit.** This block runs every time a packet leaves a queue and is about to be sent.
- `enq_ts.lookup` looks for the time we wrote at the entrance. If there is none (`!tsp`), it was not one of our packets, so stop.
- `sojourn = now - *tsp` is the key line of the whole project: time now, minus time at the entrance, equals how long this packet waited.
- `enq_ts.delete` erases the note. We are finished with this packet.

```
117      epoch = now / WINDOW_NS;
118      idx = epoch % NSLOTS;
119      s = slots.lookup(&idx);
120      if (!s)
121          return 0;
```

- Works out which tenth of a second we are in. `WINDOW_NS` is 100 million nanoseconds, which is 0.1 second.
- `idx = epoch % NSLOTS` picks one of the 64 slots. `%` means remainder, so the slot numbers go round and round like the hours on a clock.
- Then it fetches that slot from the table, and stops if that fails.

```
122      // Dequeue for one device queue is serialised by the kernel, so the slot
123      // can be updated without atomics.
124      if (s->epoch != epoch) {
125          s->epoch = epoch;
126          s->count = 0;
127          s->sum_ns = 0;
128          s->min_ns = sojourn;
129          s->max_ns = sojourn;
130      }
131      s->count += 1;
132      s->sum_ns += sojourn;
133      if (sojourn < s->min_ns)
134          s->min_ns = sojourn;
135      if (sojourn > s->max_ns)
136          s->max_ns = sojourn;
137      return 0;
138  }
```

- The comment explains why plain adding is safe here: the kernel lets only one packet at a time leave a queue, so two copies of this code never touch the same slot together.
- `if (s->epoch != epoch)` means the slot still holds an old tenth-of-a-second. In that case wipe it and start fresh, using this packet's wait as the first minimum and maximum.
- Then add this packet: count goes up by 1, and its wait is added to the total.
- The last two `if` lines keep the smallest wait seen and the largest wait seen.
- `return 0;` means "done, carry on as normal". Our program only watches. It never changes or blocks a packet.

7. controller.py - the brain

This is the biggest file. It runs as a normal program outside the kernel. Twice every second it reads the measurements, decides whether there is bufferbloat, and if so changes the queue rule.

It is built from four parts, called **classes**. A class is a bundle of data plus the actions that work on it.

Part	What it is responsible for
`Qdisc`	Talking to Linux `tc`: read the queue's counters, and replace the queue rule.
`Observer`	Loading the eBPF program and reading its tables.
`PingProbe`	Sending 5 pings per second, playing the part of the gamer who suffers.
`Controller`	The decision maker. A state machine with four moods.
`main`	Starts everything and runs the loop that ties the parts together.

The four moods (states) of the controller

```
NORMAL      everything is fine, just watching
   |  bad for 2 seconds in a row
   v
DEGRADED    bufferbloat confirmed, about to act
   |  switch to the next stronger queue rule
   v
MITIGATING  just changed something, waiting 6 seconds to see the effect
   |  delay stayed low for 3 seconds
   v
RECOVERED   it worked, keep the stronger rule
   |  link idle for 8 seconds (the download is over)
   v
back to NORMAL with the simple rule
```

Why wait, and why two different limits? Think of a thermostat that switches the heater on below 21 degrees and off above 23. If it used one number, say 22, it would click on and off every few seconds. Our controller calls a delay "bad" above 50 ms and "good" only below 20 ms. The gap between the two stops it from flipping back and forth. This gap is called **hysteresis**.

```
 1  #!/usr/bin/env python3
 2  """Bufferbloat detector + adaptive AQM controller (prototype).
 3
 4      observe  eBPF sojourn time and per-flow counters, qdisc statistics, a ping probe
 5      detect   saturated link + standing queue + RTT inflation, held for a few seconds
 6      decide   NORMAL -> DEGRADED -> MITIGATING -> RECOVERED, with hysteresis and dwell
 7      act      replace the leaf qdisc on the bottleneck: pfifo -> fq_codel -> cake
 8      verify   keep measuring; escalate or roll back if the change did not help
 9
10  Run as root. Every threshold below is an illustrative default, not a tuned value.
11  """
```

- Line 1 says "run with Python 3".
- The text between the triple quotes is a description of the program. It lists the five steps (observe, detect, decide, act, verify) and warns that the numbers used are example values, not carefully tuned ones.

```
12  import argparse
13  import collections
14  import csv
15  import json
16  import os
17  import re
18  import signal
19  import socket
20  import statistics
21  import struct
22  import subprocess
23  import sys
24  import threading
25  import time
```

- `import` borrows ready-made tools that come with Python.
- `argparse` reads options typed on the command line. `collections` gives a list that forgets old items. `csv` and `json` read and write data files. `os` and `sys` talk to the operating system. `re` finds patterns in text.
- `signal` notices when you press Ctrl-C. `socket` and `struct` turn address numbers into readable text. `statistics` gives the median. `subprocess` runs other programs. `threading` does two things at once. `time` reads the clock.

```
27  LADDER = ["pfifo", "fq_codel", "cake"]
28  WINDOW_NS = 100_000_000     # sojourn aggregation slot, matches CoDel's interval
29  NSLOTS = 64
30  HERE = os.path.dirname(os.path.abspath(__file__))
```

- `LADDER` is the list of queue rules from weakest to strongest. "Escalate" means moving one step to the right.
- `WINDOW_NS` and `NSLOTS` must match the eBPF program: 0.1 second windows, 64 slots. The underscores in `100_000_000` are only there to make the number easy to read.
- `HERE` is the folder this file is in, so we can find `observer.bpf.c` sitting next to it.

```
33  def sh(cmd):
34      return subprocess.run(cmd, check=True, capture_output=True, text=True).stdout
```

- `def` creates a helper (a function). This one runs a command and gives back whatever it printed.
- `check=True` means "raise an error if the command fails".

```
37  class Qdisc:
38      """Reads statistics from, and replaces, the leaf qdisc under the shaper."""
39
40      def __init__(self, ns, dev, parent, pfifo_limit):
41          self.tc = ["ip", "netns", "exec", ns, "tc"] if ns else ["tc"]
42          self.dev, self.parent, self.pfifo_limit = dev, parent, pfifo_limit
43          self.handle = 100
```

- The part that talks to `tc`.
- `__init__` runs once when the part is created. It prepares the beginning of every command. If the interface is inside a namespace, the command must start with `ip netns exec <name> tc`.
- It remembers the interface, the lane the queue hangs under (`1:1`), and the pfifo size.
- `self.handle = 100` is a counter used to give every new queue a fresh ID number. `self` means "this object".

```
45      def stats(self):
46          root = leaf = None
47          for q in json.loads(sh(self.tc + ["-s", "-j", "qdisc", "show", "dev", self.dev])):
48              if q.get("root"):
49                  root = q
50              elif q.get("parent") == self.parent:
51                  leaf = q
52          if root is None or leaf is None:
53              raise RuntimeError(f"no shaper + leaf qdisc on {self.dev}; run testbed.sh up first")
54          return root, leaf
```

- Asks `tc` for the counters. `-s` means statistics and `-j` means "answer in JSON", a format Python reads easily.
- The interface has two layers. The loop finds both: `root` is the speed limiter on top, and `leaf` is the queue under lane `1:1` that we manage.
- If either is missing, stop with a clear message telling you to run `testbed.sh up` first.
- It hands back both. From the root we get bytes sent. From the leaf we get how much is waiting and how much was dropped.

```
56      def apply(self, kind):
57          # A fresh handle each time: tc refuses to change a qdisc's kind in place.
58          # Note that replacing a qdisc discards whatever was queued in the old one.
59          self.handle += 1
60          args = ["limit", str(self.pfifo_limit)] if kind == "pfifo" else []
61          sh(self.tc + ["qdisc", "replace", "dev", self.dev, "parent", self.parent,
62                        "handle", f"{self.handle}:", kind] + args)
```

- **The action that changes the queue rule.** `kind` is `pfifo`, `fq_codel` or `cake`.
- It adds 1 to the handle first, because Linux will not turn an existing queue into a different kind. We must install a new one with a new ID.
- The comment is an honest warning: replacing a queue throws away the packets waiting in the old one.
- Only `pfifo` needs the extra option `limit 1000`. Then it runs `tc qdisc replace ....`

```
65  class Observer:
66      """Loads observer.bpf.c and reads its maps."""
67
68      def __init__(self, dev):
69          from bcc import BPF
70          name = dev.encode()
71          if len(name) > 15:
72              raise ValueError("interface name too long")
73          padded = name.ljust(16, b"\0")
74          mask = (b"\xff" * (len(name) + 1)).ljust(16, b"\0")
75          u64 = lambda b: f"0x{int.from_bytes(b, 'little'):x}ULL"
```

- The part that loads the eBPF program.
- `from bcc import BPF` fetches the BCC library. It is imported here, not at the top, so the program can still run in fallback mode on a machine without BCC.
- `dev.encode()` turns the interface name into raw bytes. Linux allows at most 15 letters, so anything longer is an error.
- `padded` is the name stretched to 16 bytes with zeros. `mask` marks which of those bytes actually matter.
- `u64` is a tiny helper that turns 8 bytes into one number written the way C expects (like `0x...ULL`).

```
76          self.bpf = BPF(src_file=os.path.join(HERE, "observer.bpf.c"), cflags=[
77              "-Wno-macro-redefined",
78              f"-DNAME_LO={u64(padded[:8])}", f"-DNAME_HI={u64(padded[8:])}",
79              f"-DMASK_LO={u64(mask[:8])}", f"-DMASK_HI={u64(mask[8:])}",
80              f"-DWINDOW_NS={WINDOW_NS}ULL", f"-DNSLOTS={NSLOTS}",
81          ])
82          self.slots = self.bpf["slots"]
83          self.flows = self.bpf["flows"]
84          self.next_epoch = time.monotonic_ns() // WINDOW_NS
85          self.prev_bytes = {}
```

- **Loads the eBPF program into the kernel.** BCC compiles `observer.bpf.c`, the kernel checks it is safe, and it starts running.
- The `-D...` items fill in the blanks in the C file: the name as two numbers, the two masks, the window length and the number of slots.
- `self.slots` and `self.flows` are handles to two of the kernel tables, so we can read them.
- `self.next_epoch` remembers which tenth-of-a-second we will read next. `//` means divide and drop the remainder.
- `self.prev_bytes` remembers each flow's byte count from last time, so we can work out its speed.

```
87      def sojourn(self):
88          """(min, avg, max) in ms over the 100 ms windows completed since the last call.
89
90          min is the smallest per-window minimum. A window with no departures counts
91          as zero: nothing was waiting. So min > X means that in every window, every
92          packet waited longer than X, which is CoDel's definition of a standing queue.
93          """
```

- Reads the waiting-time summaries. The text in quotes explains the idea, and it is worth reading slowly.
- It gives back three numbers in milliseconds: the minimum, the average and the maximum wait.
- **Why the minimum matters.** If even one packet in a window got through instantly, the queue emptied at some point, so it was only a short burst. If the minimum is high, **every** packet had to wait. That is a queue that never empties: a standing queue. This is the same idea the CoDel algorithm uses.

```
94          now_epoch = time.monotonic_ns() // WINDOW_NS
95          epochs = range(max(self.next_epoch, now_epoch - NSLOTS + 2), now_epoch)
96          self.next_epoch = now_epoch
97          mins, count, total, peak = [], 0, 0, 0
98          for epoch in epochs:
99              s = self.slots[epoch % NSLOTS]
100             if s.epoch != epoch or s.count == 0:
101                 mins.append(0)
102                 continue
103             mins.append(s.min_ns)
104             count += s.count
105             total += s.sum_ns
106             peak = max(peak, s.max_ns)
107         if not mins:
108             return None
109         return min(mins) / 1e6, (total / count / 1e6 if count else 0.0), peak / 1e6
```

- Works out which tenth-of-a-second is "now", then lists the finished ones we have not read yet (`epochs`).
- `max(...)` stops us from reaching back further than the 64 slots can remember.
- The loop looks at each finished window. If the slot holds a different window or has no packets, nothing was waiting, so we record a minimum of 0 and `continue` to the next one.
- Otherwise we collect its minimum, add up the counts and totals, and keep the biggest maximum.
- `if not mins: return None` means there was nothing new to read.
- The last line converts from nanoseconds to milliseconds (`/ 1e6` means divide by a million) and gives back minimum, average and maximum.

```
111      def top_flows(self, dt, n=3):
112          rates, seen = [], {}
113          for k, v in self.flows.items():
114              key = (k.saddr, k.daddr, k.sport, k.dport, k.proto)
115              seen[key] = v.bytes
116              delta = v.bytes - self.prev_bytes.get(key, 0)
117              if delta > 0:
118                  rates.append((delta * 8 / dt / 1e6, key))
119          self.prev_bytes = seen
120          ip = lambda a: socket.inet_ntoa(struct.pack("I", a))
121          proto = {1: "icmp", 6: "tcp", 17: "udp"}
122          return len(seen), [
123              f"{proto.get(k[4], k[4])} {ip(k[0])}:{k[2]}>{ip(k[1])}:{k[3]} {mbit:.1f}Mbit"
124              for mbit, k in sorted(rates, reverse=True)[:n]]
```

- Finds out which flows are sending the most right now. This tells us who is causing the traffic jam.
- For every flow in the kernel table: take its byte count now, subtract the count from last time, and turn the difference into a speed in Mbit/s (`* 8` turns bytes into bits, `/ dt` divides by the seconds passed).
- `ip` is a tiny helper that turns an address number into text like `10.0.1.2`. `proto` turns type numbers into names: 1 is ping (icmp), 6 is tcp, 17 is udp.
- It gives back how many flows exist and a readable line for each of the fastest `n` (3 unless told otherwise).

```
127  class PingProbe(threading.Thread):
128      """Interactive 'victim' flow: 5 pings/s across the bottleneck."""
129
130      def __init__(self, ns, target, t0, log_path, baseline_window):
131          super().__init__(daemon=True)
132          cmd = ["ping", "-n", "-i", "0.2", target]
133          self.proc = subprocess.Popen((["ip", "netns", "exec", ns] if ns else []) + cmd,
134                                       stdout=subprocess.PIPE, stderr=subprocess.DEVNULL, text=True)
135          self.t0, self.lock, self.fresh = t0, threading.Lock(), []
136          self.history = collections.deque()
137          self.baseline_window = baseline_window
138          self.log = open(log_path, "w", newline="")
139          self.writer = csv.writer(self.log)
140          self.writer.writerow(["t", "rtt_ms"])
```

- The "gamer". It is a **thread**, which means it runs in the background at the same time as the main loop.
- `ping -n -i 0.2` sends one ping every 0.2 seconds (5 per second). `-n` means "do not look up names".
- `subprocess.Popen` starts ping and lets us read what it prints, line by line.
- `self.lock` is like a "one at a time" sign, so the two threads do not edit the same list at the same moment.
- `self.fresh` holds new results the main loop has not seen yet. `self.history` holds recent results for finding the baseline.
- It also opens a file and writes the header row `t, rtt_ms`, so every single ping is saved for the graph.

```
142      def run(self):
143          for line in self.proc.stdout:
144              m = re.search(r"time=([\d.]+) ms", line)
145              if not m:
146                  continue
147              t, rtt = time.monotonic() - self.t0, float(m.group(1))
148              self.writer.writerow([f"{t:.3f}", rtt])
149              with self.lock:
150                  self.fresh.append(rtt)
151                  self.history.append((t, rtt))
```

- The background loop. For every line ping prints, it searches for the pattern `time=... ms`.
- If the line has no time (for example a header line), skip it with `continue`.
- Otherwise it notes the time since the start and the ping value, writes them to the file, and adds them to both lists while holding the lock.

```
153      def sample(self, now):
154          """(median RTT since last call, baseline = windowed minimum RTT), or (None, baseline)."""
155          with self.lock:
156              fresh, self.fresh = self.fresh, []
157              while self.history and self.history[0][0] < now - self.baseline_window:
158                  self.history.popleft()
159              base = min((r for _, r in self.history), default=None)
160          return (statistics.median(fresh) if fresh else None), base
```

- Called by the main loop twice a second.
- It takes the fresh results and empties that list in one move.
- It throws away history older than the baseline window (120 seconds unless told otherwise).
- `base` is the smallest ping seen in that time. That is our **baseline**: what the ping looks like when nothing is queueing.
- It gives back the median (middle value) of the fresh pings, and the baseline. If there were no fresh pings, it gives back `None`, which means "nothing".

```
162      def stop(self):
163          self.proc.terminate()
164          self.log.flush()
```

- Stops ping and makes sure everything is written to the file.

```
167  class Controller:
168      def __init__(self, args, qdisc, emit):
169          self.a, self.qdisc, self.emit = args, qdisc, emit
170          self.state = "NORMAL"
171          self.rung = LADDER.index(args.base)
172          self.policy = args.base
173          self.recent = collections.deque(maxlen=64)   # (t, delay_ms)
174          self.bad_since = self.good_since = self.idle_since = None
175          self.changed_at = self.locked_until = 0.0
176          self.before = None
177          self.prev_rung = self.rung
178          self.told_top = False
```

- **The decision maker.**
- It starts in the `NORMAL` mood, on the base queue rule. `self.rung` is our position on the ladder.
- `self.recent` remembers the last 64 delay readings. `maxlen=64` makes it forget older ones automatically.
- `bad_since`, `good_since` and `idle_since` are three stopwatches. Each remembers when a condition started, or `None` if it is not happening right now.
- `changed_at` is when we last changed the queue. `locked_until` is a "do not act before this time" mark. `before` is the delay just before a change, for comparing later.
- `prev_rung` is where we were before the last change, so we can go back. `told_top` stops us printing the same message again and again.

```
180      def _median_delay(self, since):
181          vals = [d for t, d in self.recent if t >= since]
182          return statistics.median(vals) if vals else 0.0
```

- A helper: the middle value of the delay readings since a given time. The median is used instead of the average so that one odd reading cannot fool us.

```
184      def _switch(self, rung, now, why):
185          old = self.policy
186          self.before = self._median_delay(now - 3)
187          self.prev_rung, self.rung, self.policy = self.rung, rung, LADDER[rung]
188          self.qdisc.apply(self.policy)
189          self.changed_at, self.told_top = now, False
190          self.bad_since = self.good_since = self.idle_since = None
191          self.emit(f"qdisc {old} -> {self.policy}  ({why})")
```

- A helper that performs a queue change and does the bookkeeping.
- It remembers the old rule, and records the typical delay over the last 3 seconds as `before`.
- It updates our ladder position and calls `self.qdisc.apply`, which runs the real `tc` command.
- It notes the time of the change and resets the three stopwatches, because old timings mean nothing after a change.
- `self.emit` prints a message such as `qdisc pfifo -> fq_codel` and records it in the log.

```
193      def _goto(self, state, why):
194          self.emit(f"state {self.state} -> {state}  ({why})")
195          self.state = state
```

- A helper that changes mood and prints why, for example `state NORMAL -> DEGRADED`.

```
197      def step(self, now, rate_mbit, delay_ms, inflation_ms):
198          a = self.a
199          self.recent.append((now, delay_ms))
200          saturated = rate_mbit >= a.saturation * a.rate_mbit
201          # The two thresholds are deliberately far apart (hysteresis).
202          bad = saturated and delay_ms > a.enter_ms and (inflation_ms is None or inflation_ms > a.enter_ms)
203          good = delay_ms < a.exit_ms and (inflation_ms is None or inflation_ms < a.exit_ms)
204          idle = rate_mbit < 0.5 * a.rate_mbit
```

- **Called twice a second with the newest measurements. The whole decision lives here.**
- `a = self.a` is just a short name for the settings. The new delay reading is added to memory.
- `saturated`: is the link nearly full? True if the speed is at least 85% of the link's capacity.
- `bad`: all three must be true. The link is saturated, **and** the queue delay is above 50 ms, **and** the ping grew by more than 50 ms over its baseline. Three separate clues that agree.
- `good`: the delay is under 20 ms and the ping is close to its baseline.
- `idle`: the link is less than half used, so the download has stopped.
- `inflation_ms is None or ...` means "if we have no ping reading this time, do not let that block the decision".

```
206          held = lambda since: since is not None and now - since
207          self.bad_since = (self.bad_since or now) if bad else None
208          self.good_since = (self.good_since or now) if good else None
209          self.idle_since = (self.idle_since or now) if idle else None
210
211          if a.observe_only:
212              return
```

- `held` answers "how many seconds has this been going on?".
- The next three lines run the stopwatches. If the condition is true, keep the old start time or start now. If it is false, reset to `None`.
- `if a.observe_only: return` is for the comparison run: measure everything, but never touch the queue.

```
214          if self.state == "NORMAL":
215              if now >= self.locked_until and held(self.bad_since) >= a.persist:
216                  self._goto("DEGRADED", f"standing queue for {a.persist:g}s: delay {delay_ms:.0f} ms on a saturat
       > ed link")
```

- In the NORMAL mood: if we are not locked out and things have been bad for 2 seconds in a row (`a.persist`), move to DEGRADED.
- The 2-second rule matters. A short burst is normal and harmless. Only a queue that **stays** long is bufferbloat.

```
218          elif self.state == "DEGRADED":
219              if self.rung + 1 < len(LADDER):
220                  self._switch(self.rung + 1, now, "escalate")
221                  self._goto("MITIGATING", f"holding for {a.dwell:g}s dwell")
222              else:
223                  self._goto("MITIGATING", "already on the strongest policy")
224                  self.changed_at = now
```

- In the DEGRADED mood: act.
- If there is a stronger rule on the ladder, switch to it and move to MITIGATING.
- If we are already on the strongest rule, there is nothing more to try. Move to MITIGATING anyway and restart the waiting clock.

```
226          elif self.state == "MITIGATING":
227              if now - self.changed_at < a.dwell:
228                  return      # measurements still reflect the transient of our own change
229              after = self._median_delay(self.changed_at + a.dwell / 2)
230              if held(self.good_since) >= a.exit_persist:
231                  self._goto("RECOVERED", f"delay {self.before:.0f} -> {after:.0f} ms, keeping {self.policy}")
232              elif self.before and after > 1.25 * self.before and self.rung != self.prev_rung:
233                  self._switch(self.prev_rung, now, f"rollback: delay got worse, {self.before:.0f} -> {after:.0f}
       > ms")
234                  self.locked_until = now + a.cooldown
235                  self._goto("NORMAL", f"locked out for {a.cooldown:g}s")
236              elif held(self.bad_since) >= a.persist:
237                  if self.rung + 1 < len(LADDER):
238                      self._switch(self.rung + 1, now, f"{LADDER[self.rung]} did not help, escalate")
239                  elif not self.told_top:
240                      self.emit("top of the ladder and still degraded; holding")
241                      self.told_top = True
```

- In the MITIGATING mood: first, wait. If fewer than 6 seconds (`a.dwell`) have passed since the change, `return` and do nothing. Right after a change the numbers are still settling, so judging now would be unfair.
- After the wait, `after` is the typical delay since the halfway point of the wait.
- **It worked:** if things have been good for 3 seconds, move to RECOVERED and keep the new rule.
- **It got worse:** if the delay is now more than 1.25 times what it was before, undo the change (rollback), lock ourselves out for 8 seconds, and go back to NORMAL.
- **Still bad:** if things have been bad for 2 seconds, climb one more step. If we are already at the top, say so once and keep holding.

```
243         elif self.state == "RECOVERED":
244             if held(self.bad_since) >= a.persist:
245                 self._goto("DEGRADED", f"degraded again under {self.policy}")
246             elif held(self.idle_since) >= a.cooldown and not a.no_revert and self.policy != a.base:
247                 # Revert on absence of load, never on low delay: the low delay is
248                 # caused by the policy, so reverting on it would flap.
249                 self._switch(LADDER.index(a.base), now, f"link idle for {a.cooldown:g}s, back to base")
250                 self._goto("NORMAL", "load gone")
```

- In the RECOVERED mood: if it turns bad again for 2 seconds, go back to DEGRADED and try something stronger.
- If the link has been idle for 8 seconds (`a.cooldown`), the download is over. Put the simple rule back and return to NORMAL.
- The comment explains an important choice. We go back only when the **load is gone**, never just because the delay is low. The delay is low **because** of our fix. Removing the fix for that reason would bring the problem straight back, and we would flip back and forth forever.

```
253  def main():
254      p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
255      p.add_argument("--ns", default="bb-r", help="namespace of the bottleneck router ('' for none)")
256      p.add_argument("--dev", default="r-eth1", help="bottleneck egress interface")
257      p.add_argument("--parent", default="1:1", help="shaper class holding the managed leaf qdisc")
258      p.add_argument("--rate-mbit", type=float, default=10.0, help="bottleneck capacity")
259      p.add_argument("--pfifo-limit", type=int, default=1000)
260      p.add_argument("--base", choices=LADDER, default="pfifo", help="policy when nothing is wrong")
261      p.add_argument("--probe-ns", default="bb-h1")
262      p.add_argument("--probe-target", default="10.0.2.2")
263      p.add_argument("--tick", type=float, default=0.5, help="control loop period, s")
264      p.add_argument("--enter-ms", type=float, default=50, help="upper threshold: delay above this is bad")
265      p.add_argument("--exit-ms", type=float, default=20, help="lower threshold: delay below this is good")
266      p.add_argument("--persist", type=float, default=2, help="s a bad condition must last before acting")
267      p.add_argument("--exit-persist", type=float, default=3, help="s a good condition must last")
268      p.add_argument("--dwell", type=float, default=6, help="s to hold after a change before judging it")
269      p.add_argument("--cooldown", type=float, default=8, help="s of idle link before reverting to base")
270      p.add_argument("--saturation", type=float, default=0.85, help="fraction of capacity that counts as saturated
     > ")
271      p.add_argument("--baseline-window", type=float, default=120, help="s over which min RTT is the baseline")
272      p.add_argument("--observe-only", action="store_true", help="measure and detect, never change the qdisc")
273      p.add_argument("--no-revert", action="store_true", help="keep the stronger policy after load ends")
274      p.add_argument("--no-ebpf", action="store_true", help="fallback: derive delay from backlog / rate")
275      p.add_argument("--no-probe", action="store_true", help="do not run the ping probe")
276      p.add_argument("--duration", type=float, default=0, help="stop after this many seconds (0 = until Ctrl-C)")
277      p.add_argument("--log", default="results/run.csv")
278      args = p.parse_args()
```

- Where the program starts. This first block defines every option you can type on the command line. Each `add_argument` line gives a name, a default value and a short help text.
- `--ns`, `--dev`, `--parent`: which pretend computer, which interface, and which lane holds the queue.
- `--rate-mbit`: the link's capacity. `--pfifo-limit`: the size of the simple queue. `--base`: the rule to use when all is well.
- `--probe-ns`, `--probe-target`: where the ping runs from, and who it pings.
- `--tick`: how often the brain thinks (every 0.5 seconds).
- `--enter-ms` (50) and `--exit-ms` (20): the bad and good limits. `--persist` (2 s), `--exit-persist` (3 s), `--dwell` (6 s) and `--cooldown` (8 s): the waiting times described above.
- `--saturation` (0.85): how full counts as full. `--baseline-window` (120 s): how far back to look for the smallest ping.
- `--observe-only`: watch but never act. `--no-revert`: keep the strong rule even after the load ends. `--no-ebpf`: run without eBPF. `--no-probe`: run without the ping.
- `--duration`: stop by itself after this many seconds. `--log`: where to save the numbers.
- `args = p.parse_args()` reads what you actually typed.

```
280     if os.geteuid() != 0:
281         sys.exit("run as root (loading eBPF and changing qdiscs both need it)")
282     os.makedirs(os.path.dirname(args.log) or ".", exist_ok=True)
```

- Checks that we are the administrator (user number 0). Loading eBPF and changing queues both need that. If not, stop with a message.
- `os.makedirs` creates the folder for the log file if it does not exist yet.

```
284     qdisc = Qdisc(args.ns, args.dev, args.parent, args.pfifo_limit)
285     qdisc.stats()
286     qdisc.apply(args.base)
```

- Creates the `Qdisc` part. Calling `stats()` once is a test: if the testbed is not up, we stop here with a clear message.
- `qdisc.apply(args.base)` installs the base rule, so every run begins from the same known state.

```
288     observer = None
289     if not args.no_ebpf:
290         try:
291             observer = Observer(args.dev)
292         except Exception as e:      # noqa: BLE001 - any load failure means fallback
293             print(f"!! eBPF observer failed to load ({e}); falling back to backlog/rate", flush=True)
```

- Tries to load the eBPF program, unless `--no-ebpf` was given.
- `try` and `except` mean "try this, and if anything goes wrong, do the other thing instead of crashing". If loading fails, print a warning and carry on without eBPF.
- In that fallback mode the delay is estimated from the queue length. It is less exact, but the program still works.

```
295     t0 = time.monotonic()
296     probe = None
297     if not args.no_probe:
298         probe = PingProbe(args.probe_ns, args.probe_target, t0, args.log.replace(".csv", "") + ".ping.csv",
299                           args.baseline_window)
300         probe.start()
```

- `t0` is the start time. All times in the log are counted from here.
- Unless `--no-probe` was given, create the ping probe and start it in the background. Its results go to a file with the same name as the log, ending in `.ping.csv`.

```
302     events = []
303     def emit(msg):
304         events.append(msg)
305         print(f"  >>> {msg}", flush=True)
306
307     ctl = Controller(args, qdisc, emit)
308     stop = threading.Event()
309     signal.signal(signal.SIGINT, lambda *_: stop.set())
310     signal.signal(signal.SIGTERM, lambda *_: stop.set())
```

- `emit` is the helper that prints the `>>>` messages and remembers them for the log.
- Then the `Controller` is created.
- `stop` is a flag. The two `signal.signal` lines say: when Ctrl-C is pressed or the program is asked to end, raise the flag, so the loop finishes cleanly instead of being killed halfway.

```
312     fields = ["t", "state", "policy", "rate_mbit", "backlog_bytes", "qdelay_derived_ms", "sojourn_min_ms",
313               "sojourn_avg_ms", "sojourn_max_ms", "rtt_ms", "rtt_base_ms", "inflation_ms", "drops", "flows", "ev
    > ent"]
314     mode = "observe-only" if args.observe_only else "adaptive"
315     source = "eBPF sojourn (min per 100 ms window)" if observer else "backlog / rate (no eBPF)"
316     print(f"{mode} controller on {args.dev}: {args.rate_mbit:g} Mbit, base {args.base}, delay signal: {source}")
317     print(f"{'t':>6} {'state':<10} {'qdisc':<8} {'rate':>6} {'backlog':>8} {'derived':>8} {'sojourn':>8} "
318           f"{'rtt':>7} {'infl':>6} {'drops':>6}  top flow")
```

- The column names for the log file.
- `mode` and `source` are only text for the first printed line: are we adaptive or observe-only, and where does the delay reading come from.
- The two `print` lines show that banner and the column headings of the live table. Codes like `:>6` mean "right-aligned, 6 characters wide", which keeps the columns straight.

```
320     with open(args.log, "w", newline="") as f:
321         w = csv.DictWriter(f, fieldnames=fields)
322         w.writeheader()
323         root, _ = qdisc.stats()
324         prev_bytes, prev_t, drops, prev_leaf = root["bytes"], t0, 0, (qdisc.handle, 0)
```

- Opens the log file and writes the heading row.
- Before the loop starts it reads the counters once, so the first speed calculation has a starting point.
- `prev_bytes` and `prev_t` are the bytes and time from last round. `drops` is the running total of dropped packets. `prev_leaf` remembers which queue we last read and its drop counter.

```
325         while not stop.wait(args.tick):
326             now = time.monotonic()
327             t, dt = now - t0, now - prev_t
328             if args.duration and t > args.duration:
329                 break
330             root, leaf = qdisc.stats()
331             rate = (root["bytes"] - prev_bytes) * 8 / dt / 1e6
332             prev_bytes, prev_t = root["bytes"], now
333             # Leaf counters restart whenever the leaf is replaced.
334             if prev_leaf[0] == qdisc.handle:
335                 drops += max(0, leaf["drops"] - prev_leaf[1])
336             else:
337                 drops += leaf["drops"]
338             prev_leaf = (qdisc.handle, leaf["drops"])
339             derived = leaf["backlog"] * 8 / (args.rate_mbit * 1e6) * 1e3
```

- **The main loop.** `stop.wait(0.5)` waits half a second. It repeats until the stop flag is raised.
- `t` is seconds since the start, and `dt` is seconds since the last round. If a duration was given and it has passed, `break` leaves the loop.
- `rate` is the current speed: bytes sent since last round, times 8 for bits, divided by the time, divided by a million for Mbit/s.
- The drop counter needs care. When we replace the queue, its counter restarts at zero. So if the queue is the same one as last time we add the difference, and if it is a new one we add its whole counter.
- `derived` is the estimated delay: bytes waiting, turned into bits, divided by the link speed. This is the cheap estimate, to compare with the real eBPF measurement.

```
341             soj = observer.sojourn() if observer else None
342             nflows, top = observer.top_flows(dt) if observer else (0, [])
343             rtt, base = probe.sample(t) if probe else (None, None)
344             inflation = rtt - base if rtt is not None and base is not None else None
345             delay = soj[0] if soj else derived
```

- Collects the three kinds of measurement for this round.
- `soj` is the real waiting time from eBPF. `nflows` and `top` are the flow count and the busiest flows. `rtt` and `base` are the ping and its baseline.
- `inflation = rtt - base` is how much the ping has grown. If the ping normally takes 20 ms and now takes 200 ms, the inflation is 180 ms, and all of it is queueing.
- `delay` is the number the brain will use: the eBPF minimum if we have it, otherwise the estimate.

```
347             n_events = len(events)
348             ctl.step(t, rate, delay, inflation)
349             fmt = lambda v, nd=1: "" if v is None else f"{v:.{nd}f}"
350             w.writerow({
351                 "t": f"{t:.2f}", "state": ctl.state, "policy": ctl.policy, "rate_mbit": f"{rate:.2f}",
352                 "backlog_bytes": leaf["backlog"], "qdelay_derived_ms": f"{derived:.1f}",
353                 "sojourn_min_ms": fmt(soj[0] if soj else None), "sojourn_avg_ms": fmt(soj[1] if soj else None),
354                 "sojourn_max_ms": fmt(soj[2] if soj else None), "rtt_ms": fmt(rtt), "rtt_base_ms": fmt(base),
355                 "inflation_ms": fmt(inflation), "drops": drops, "flows": nflows,
356                 "event": " | ".join(events[n_events:]),
357             })
358             f.flush()
359             ms = lambda v: "-" if v is None else f"{v:.0f}ms"
360             print(f"{t:6.1f} {ctl.state:<10} {ctl.policy:<8} {rate:5.1f}M {leaf['backlog'] // 1000:6d}kB "
361                   f"{ms(derived):>8} {ms(soj[0] if soj else None):>8} {ms(rtt):>7} {ms(inflation):>6} {drops:6d}
      >   "
362                   f"{top[0] if top else ''}", flush=True)
```

- Notes how many messages existed, so we can tell which ones are new after the next line.
- `ctl.step(...)` hands the measurements to the brain. **This single line is where the decision happens.**
- `fmt` is a small helper that writes a number with one decimal place, or leaves the cell empty if there is no value.
- `w.writerow({...})` saves one row to the log: time, mood, queue rule, speed, bytes waiting, both delay figures, ping, baseline, inflation, drops, flow count and any new messages.
- `f.flush()` forces the row onto the disk at once, so nothing is lost if the program is stopped.
- `ms` is a helper for the screen: a whole number followed by `ms`, or a dash if there is no value.
- The final `print` shows one line of the live table that you watch during the demo.

```
364     if probe:
365         probe.stop()
366     if ctl.policy != args.base:
367         qdisc.apply(args.base)      # leave the interface in a known state
368         print(f"  >>> exit: restored {args.base}")
```

- After the loop ends: stop the ping.
- If we are not on the base rule, put the base rule back and say so. A good tool leaves the system the way it found it.

```
371 if __name__ == "__main__":
372     main()
```

- Standard Python ending. It means "if this file is being run directly, start `main()`".

8. run_demo.sh - run the experiment

A fair test changes only one thing. So this file runs exactly the same download twice. The first time the brain only watches. The second time it is allowed to act. Any difference between the two runs is then caused by the brain.

```
 1  #!/usr/bin/env bash
 2  # Runs the same scripted load twice on a fresh testbed:
 3  #   1. static_pfifo  controller observes but never acts   (the control condition)
 4  #   2. adaptive      controller detects and switches the qdisc
 5  # then prints a comparison and writes results/demo.png.
 6  #
 7  # Usage: sudo ./run_demo.sh            both runs
 8  #        sudo ./run_demo.sh adaptive   just the adaptive run (good for a live demo)
```

- Line 1 says "run with bash". The comments describe the two runs and show how to use the file. With no extra word it does both runs. With `adaptive` it does only the second one.

```
 9  set -euo pipefail
10  cd "$(dirname "$0")"
11  [ "$(id -u)" = 0 ] || { echo "run with sudo" >&2; exit 1; }
```

- The safety switch again: stop at the first error.
- `cd "$(dirname "$0")"` moves into the folder this file is in, so it works no matter where you start it from.
- The third line checks we are the administrator. `id -u` prints our user number, and 0 is the administrator. `||` means "otherwise": print `run with sudo` and stop.

```
13  IDLE="${IDLE:-8}"       # quiet seconds before the bulk flow starts
14  LOAD="${LOAD:-30}"      # seconds of bulk TCP
15  TAIL="${TAIL:-14}"      # quiet seconds after it ends
16  OUT=results
17  mkdir -p "$OUT"
```

- The timetable of one run: 8 quiet seconds, then 30 seconds of download, then 14 quiet seconds.
- The quiet start lets the ping find its baseline. The quiet end lets us watch the controller notice the load is gone.
- `OUT=results` is the folder for the output, and `mkdir -p` creates it if it is missing.

```
19  run() {
20      local name=$1; shift
21      echo; echo "=== $name: ${IDLE}s idle, ${LOAD}s bulk TCP h1 -> h2, ${TAIL}s idle ==="
22      ./testbed.sh up
23      ip netns exec bb-h2 iperf3 -s -1 >/dev/null 2>&1 &
24      ( sleep "$IDLE"; ip netns exec bb-h1 iperf3 -c 10.0.2.2 -t "$LOAD" -J > "$OUT/$name.iperf.json" ) &
25      python3 controller.py --duration $((IDLE + LOAD + TAIL)) --log "$OUT/$name.csv" "$@"
26      wait
27  }
```

- A helper that does one complete run. `name` is what to call this run. `shift` removes that word, so whatever is left are extra options for the controller.
- It prints a heading, then builds a fresh network with `./testbed.sh up`.
- It starts an `iperf3` server on h2. `-s` means server and `-1` means "serve one client, then quit". The `&` at the end means "run in the background and keep going".
- The line in round brackets is the download, also in the background. It sleeps for the 8 quiet seconds, then h1 sends to h2 for 30 seconds. `-J` saves the result as JSON.
- Then the controller runs in the front, so you see its live table. `$((IDLE + LOAD + TAIL))` adds the three times to get 52 seconds. `"$@"` passes on the extra options.
- `wait` pauses until the background jobs have finished.

```
case "${1:-both}" in
    both)     run static_pfifo --observe-only; run adaptive ;;
    static)   run static_pfifo --observe-only ;;
    adaptive) run adaptive ;;
    *) echo "usage: $0 [both|static|adaptive]" >&2; exit 1 ;;
esac
```

- Chooses what to do based on the word you typed. `${1:-both}` means "use `both` if nothing was typed".
- `both` runs `static_pfifo` with `--observe-only` (brain off), then runs `adaptive` (brain on).
- `static` and `adaptive` each do just one of the two. Anything else prints the usage line and stops.

```
./testbed.sh down
if python3 -c "import matplotlib" 2>/dev/null; then
    python3 plot.py "$OUT"
else
    echo "matplotlib is not installed here; run 'python3 plot.py $OUT' on the host for the summary and plot"
fi
```

- Cleans up the pretend network.
- Then it checks whether the graph tool (matplotlib) is installed here. If yes, it makes the table and the graph. If not, it tells you the command to run on a machine that has it.

9. plot.py - turn numbers into a picture

The controller saved its numbers in files. This file reads them, prints a comparison table, and draws the graph shown in section 10.

```
#!/usr/bin/env python3
"""Summarise and plot the runs in a results directory (default: results/)."""
import csv
import json
import os
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
```

- "Run with Python 3", and a one-line description.
- The imports: `csv` and `json` read the data files, `os` and `sys` deal with file names and typed options.
- `matplotlib` is the drawing library. `matplotlib.use("Agg")` means "draw straight into a file, do not open a window". `numpy` (called `np`) does fast maths on lists of numbers.

```
RUNS = [("static_pfifo", "Static pfifo (controller off)"), ("adaptive", "Adaptive controller")]
BLUE, ORANGE = "#2a78d6", "#eb6834"
INK, MUTED, GRID = "#0b0b0b", "#52514e", "#e4e3df"
FLOOR_MS = 0.5      # log axis: anything faster is drawn here
```

- `RUNS` lists the two runs: the file name of each and the title to show.
- The next two lines are colours written as codes: blue and orange for the lines, and three greys for text and the grid.
- `FLOOR_MS = 0.5`: the graph uses a scale that cannot show zero, so anything smaller than 0.5 ms is drawn at 0.5.

```
def load(out, name):
    path = os.path.join(out, name + ".csv")
    if not os.path.exists(path):
        return None
    rows = list(csv.DictReader(open(path)))
    num = lambda r, k: float(r[k]) if r[k] else np.nan
    run = {k: np.array([num(r, k) for r in rows]) for k in
           ("t", "rate_mbit", "sojourn_avg_ms", "qdelay_derived_ms", "drops")}
    run["policy"] = [r["policy"] for r in rows]
    run["changes"] = [(run["t"][i], rows[i]["policy"]) for i in range(1, len(rows))
                      if rows[i]["policy"] != rows[i - 1]["policy"]]
```

- Reads all the files for one run. If the main file is missing, it gives back `None` (nothing).
- `rows` is every line of the log. `num` turns a piece of text into a number, or into `nan` ("not a number") if the cell is empty.
- `run` becomes a bundle holding a list of numbers for each column we need: time, speed, average waiting time, estimated delay and drops.
- `run["policy"]` is which queue rule was active on each row.
- `run["changes"]` finds the moments where the rule is different from the row before. Those are the switches.

```
    ping = list(csv.DictReader(open(os.path.join(out, name + ".ping.csv"))))
    run["ping_t"] = np.array([float(r["t"]) for r in ping])
    run["ping"] = np.array([float(r["rtt_ms"]) for r in ping])
```

- Reads the ping file: one list for the times and one for the ping values.

```
33      try:
34          end = json.load(open(os.path.join(out, name + ".iperf.json")))["end"]
35          run["goodput"] = end["sum_received"]["bits_per_second"] / 1e6
36          run["retrans"] = end["sum_sent"].get("retransmits", 0)
37      except (OSError, KeyError, ValueError):
38          run["goodput"] = run["retrans"] = np.nan
```

- Reads the download tool's own report. `goodput` is the speed the download really achieved, in Mbit/s. `retrans` is how many packets had to be sent again.
- If that file is missing or damaged, `except` catches the problem and stores `nan` instead of crashing.

```
39      busy = run["t"][run["rate_mbit"] > 0.5 * np.nanmax(run["rate_mbit"])]
40      run["load"] = (busy.min(), busy.max()) if len(busy) else (0, 0)
41      return run
```

- Finds when the download was running: the rows where the speed was above half of the highest speed seen.
- `run["load"]` stores the first and last such moment. The table uses this to look only at pings during the download.
- `return run` hands the bundle back.

```
44  def summary(runs):
45      lines = ["Interactive probe RTT while the bulk flow is running (ms), and what it cost",
46               f"{'run':<30}{'p50':>8}{'p95':>8}{'p99':>8}{'goodput':>10}{'retrans':>9}{'drops':>7}{'switches':>10
      > }"]
47      for label, r in runs:
48          lo, hi = r["load"]
49          rtt = r["ping"][(r["ping_t"] >= lo) & (r["ping_t"] <= hi)]
50          p = np.percentile(rtt, [50, 95, 99]) if len(rtt) else [np.nan] * 3
51          lines.append(f"{label:<30}{p[0]:8.1f}{p[1]:8.1f}{p[2]:8.1f}{r['goodput']:8.2f} M{r['retrans']:9.0f}"
52                       f"{r['drops'][-1]:7.0f}{len(r['changes']):10d}")
53      return "\n".join(lines)
```

- Builds the comparison table as text. The first two lines are the title and the column headings.
- For each run it picks the pings that happened during the download (between `lo` and `hi`).
- `np.percentile(rtt, [50, 95, 99])` gives three numbers. **p50** is the middle ping, a typical one. **p95** means 95 out of 100 pings were faster than this. **p99** means 99 out of 100 were faster. The high ones show the worst moments, which is what ruins a game or a call.
- It then adds one row with those three numbers plus the download speed, resends, total drops and the number of queue switches.
- The last line joins all rows into one piece of text.

```
56  def plot(runs, path):
57      plt.rcParams.update({"font.size": 9, "axes.edgecolor": GRID, "axes.labelcolor": MUTED,
58                           "xtick.color": MUTED, "ytick.color": MUTED, "axes.titlecolor": INK,
59                           "axes.spines.top": False, "axes.spines.right": False})
60      fig, axes = plt.subplots(2, len(runs), figsize=(5.6 * len(runs), 5.6), sharex="col", sharey="row",
61                               squeeze=False, gridspec_kw={"height_ratios": [2.2, 1]})
```

- Draws the picture. `plt.rcParams.update` sets the general look: font size, grey axes, no box lines on the top and right.
- `plt.subplots(2, ...)` makes a grid of small graphs with 2 rows and one column per run. `sharey="row"` gives graphs in the same row the same scale, so they can be compared fairly side by side.
- `height_ratios` makes the top row about twice as tall as the bottom row.

```
    for col, (label, r) in enumerate(runs):
        top, bottom = axes[0][col], axes[1][col]
        top.plot(r["ping_t"], np.maximum(r["ping"], FLOOR_MS), color=BLUE, lw=1.5, label="Ping RTT, h1 to h2")
        top.plot(r["t"], np.maximum(r["sojourn_avg_ms"], FLOOR_MS), color=ORANGE, lw=1.5,
                 label="Queue sojourn time, eBPF (mean per 0.5 s)")
        top.set_yscale("log")
        top.set_title(label, loc="left", fontweight="bold")
        bottom.plot(r["t"], r["rate_mbit"], color=BLUE, lw=1.5)
        bottom.set_ylim(bottom=0)
        bottom.set_xlabel("time (s)")
```

- Repeats once per run. `top` and `bottom` are that run's two graphs.
- On the top graph it draws the ping in blue and the eBPF waiting time in orange. `np.maximum(..., FLOOR_MS)` lifts tiny values to 0.5 so the scale can show them.
- `set_yscale("log")` uses a scale where each step up is 10 times bigger (1, 10, 100, 1000). That lets 20 ms and 1000 ms both be seen clearly on one graph.
- It sets the title, then draws the link speed on the bottom graph, makes that graph start at zero, and labels the time axis.

```
        for ax in (top, bottom):
            ax.grid(True, color=GRID, lw=0.6)
            ax.set_axisbelow(True)
            for t, _ in r["changes"]:
                ax.axvline(t, color=MUTED, lw=0.8, ls=(0, (4, 3)))
        for t, policy in r["changes"]:
            top.annotate(f"qdisc: {policy}", (t, 1), xycoords=("data", "axes fraction"), xytext=(3, -10),
                         textcoords="offset points", color=INK, fontsize=8)
```

- For both graphs: draw a faint grid behind the data, and a dashed vertical line at each moment the queue rule changed.
- The second loop writes a small label next to each dashed line on the top graph, such as `qdisc: fq_codel`.

```
    axes[0][0].set_ylabel("delay (ms, log scale)")
    axes[1][0].set_ylabel("bottleneck egress (Mbit/s)")
    fig.legend(*axes[0][0].get_legend_handles_labels(), loc="upper right", ncol=2, frameon=False)
    fig.suptitle("Latency under load at a 10 Mbit bottleneck: same traffic, controller off vs on",
                 x=0.01, ha="left", color=INK, fontsize=11)
    fig.tight_layout()
    fig.savefig(path, dpi=150, facecolor="#fcfcfb")
```

- Labels the two vertical axes: delay in milliseconds, and speed in Mbit/s.
- `fig.legend` adds the key that says which colour is which line.
- `fig.suptitle` adds the big title. `tight_layout` tidies the spacing. `savefig` writes the picture to a file.

```
def main():
    out = sys.argv[1] if len(sys.argv) > 1 else "results"
    runs = [(label, r) for name, label in RUNS if (r := load(out, name))]
    if not runs:
        sys.exit(f"no runs found in {out}/")
    text = summary(runs)
    print("\n" + text)
    open(os.path.join(out, "summary.txt"), "w").write(text + "\n")
    plot(runs, os.path.join(out, "demo.png"))
    print(f"\nplot: {out}/demo.png")
```

- Where the file starts. `out` is the folder to read: the one you typed, or `results` if you typed nothing.
- `runs` collects every run whose files exist. If there are none, stop with a message.
- Then it builds the table, prints it, saves it as `summary.txt`, draws the graph as `demo.png`, and prints where the graph was saved.

```
101  if __name__ == "__main__":
102      main()
```

- "If this file is run directly, start `main()`."

10. What happened when we ran it

We ran the demo once on a real Linux kernel. Both runs used the same 30-second download over the same 10 Mbit/s link.



How to read the picture

- **Left side, brain off.** When the download starts at about 8 seconds, the ping (blue) climbs from 20 ms to over 1000 ms and stays there until the download ends. The orange line, our eBPF waiting time, follows it almost exactly. That shows the extra ping really is time spent in the queue.
- **Right side, brain on.** The ping starts to climb in the same way. About 3 seconds after the download starts, the controller switches to `fq_codel` (first dashed line) and the ping drops back to about 20 ms while the download is still running.
- **Bottom graphs.** The download speed is the same in both runs, close to 10 Mbit/s. We did not fix the ping by slowing the download down.
- **Second dashed line.** After the download ends and the link has been quiet for 8 seconds, the controller puts the simple rule back.

The numbers

Run	Typical ping (p50)	p95	p99	Download speed
Brain off (static pfifo)	953.5 ms	1252.0 ms	1262.0 ms	9.32 Mbit/s
Brain on (adaptive)	21.1 ms	202.6 ms	259.0 ms	9.22 Mbit/s

The p95 and p99 of the adaptive run are still around 200 ms. Those come from the roughly 3 seconds between the download starting and the switch, when the queue was already growing.

What the controller printed

```
>>> state NORMAL -> DEGRADED  (standing queue for 2s: delay 192 ms on a saturated link)
>>> qdisc pfifo -> fq_codel  (escalate)
>>> state DEGRADED -> MITIGATING  (holding for 6s dwell)
>>> state MITIGATING -> RECOVERED  (delay 159 -> 0 ms, keeping fq_codel)
>>> qdisc fq_codel -> pfifo  (link idle for 8s, back to base)
>>> state RECOVERED -> NORMAL  (load gone)
```

The live table columns

Column	Meaning
`t`	Seconds since the controller started.
`state`	The controller's mood: NORMAL, DEGRADED, MITIGATING or RECOVERED.
`qdisc`	The queue rule installed right now.
`rate`	How fast data is leaving the slow link, in Mbit/s.
`backlog`	How much data is waiting in the queue, in kilobytes.
`derived`	Estimated waiting time, worked out from the backlog.
`sojourn`	Real waiting time measured by eBPF (the smallest wait in each 0.1 s window).
`rtt`	The ping time right now.
`infl`	Inflation: how much bigger the ping is than its baseline.
`drops`	Total packets the queue has thrown away so far.
`top flow`	The busiest conversation right now, and its speed.

11. Why this is a prototype and not the full project

The prototype proves the idea works once, in the easiest situation. The full project has to prove the brain is actually worth having. Be honest about this when you present.

Topic	Prototype (now)	Full project (still to do)
Comparison	Only compared with plain `pfifo`.	Compare with always using `fq_codel` and always using `cake`. That is the comparison that tests our own work.
Evidence	One run of each.	Many repeated runs, reporting how much the results vary.
Situations	One download plus a ping.	Link speed that changes, many flows, flows with different delays, a flow that refuses to slow down.
Ladder	Only `pfifo` to `fq_codel` has actually run.	Reach `cake`, and test the escalate and rollback paths. That code exists but has never been triggered.
Tuning	Only switches the rule.	Also adjust the settings inside a rule (`target` and `interval`).
Limits used	Example values (50 ms, 20 ms, 2 s, 6 s, 8 s).	Explain why each value was chosen, and test what happens if it changes.
Cost	Not measured.	Measure how much processor time the eBPF program uses.
Fairness	Not measured.	Check that several flows share the link evenly.
The ping	Under `fq_codel` a ping gets served first, so it looks very good.	Also measure flows that do not get that special treatment.
eBPF hook	Kernel tracepoints, loaded with BCC.	Decide between tracepoints and a tc-BPF program, and explain the choice.

The one sentence to remember. Beating `pfifo` has been known for more than ten years. What our prototype shows is that the full loop (measure, detect, decide, adapt, verify) works automatically on a real Linux kernel. Whether that automatic brain beats simply leaving a good rule switched on all the time is the question the full project must answer.

Three things we did not invent

- **The queue rules.** `pfifo`, `fq_codel` and `cake` are part of Linux. We only choose between them.
- **eBPF.** It is a Linux feature. We wrote one small program that uses it.
- **The idea of bufferbloat.** It was named and studied by others around 2010.

What is ours: the program that measures, the rules for deciding, the safe way of switching, and the experiment that checks it.